

Punto 5.7 — Migraciones idempotentes y scripts SQL

Práctica

Ejercicio: Generar scripts SQL idempotentes para el proyecto AceriaData. Generar un script con todas las migraciones, un script con un rango de migraciones, un script de reversión y un script idempotente. Revisar los scripts y aplicarlos con sqlcmd con autenticación integrada de Windows. Integrar los scripts en un pipeline de GitHub Actions simulado. Analizar el SQL generado y medir el tiempo de generación.

Contexto del proyecto: En el punto 5.6 se estudiaron las migraciones en producción, incluyendo las estrategias de despliegue y las opciones MigrationsAssembly y MigrationsHistoryTable. En este punto se profundiza en las migraciones idempotentes y los scripts SQL. Esta técnica se usará en el punto 5.8 para las migraciones en equipos.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Infrastructure
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Infrastructure → entra en la carpeta del proyecto de infraestructura.

Paso 2: Generar un script con todas las migraciones

```
dotnet ef migrations script --startup-project ../AceriaData.Console --output migraciones_completas.sql
```

dotnet ef migrations script → genera un script SQL. --startup-project ../AceriaData.Console → indica el proyecto de inicio. --output migraciones_completas.sql → guarda el script en el archivo indicado.

Resultado esperado: se crea el archivo migraciones_completas.sql con todas las migraciones.

El SQL generado incluye la creación de todas las tablas, índices y claves foráneas. También incluye la inserción de las migraciones en el historial.

Error común: si el script no se revisa antes de aplicarlo, se pueden producir errores. Se debe revisar el script.

Paso 3: Generar un script idempotente

```
dotnet ef migrations script --idempotent --startup-project ../AceriaData.Console --output
migraciones_idempotentes.sql
```

dotnet ef migrations script → genera un script SQL. --idempotent → genera un script que se puede aplicar varias veces. --startup-project ../AceriaData.Console → indica el proyecto de inicio. --output migraciones_idempotentes.sql → guarda el script en el archivo indicado.

Resultado esperado: se crea migraciones_idempotentes.sql con el script generado por EF Core. La idempotencia se basa en consultar el historial de migraciones y condicionar los bloques que aún no están aplicados; no se debe resumir como una regla manual de IF NOT EXISTS antes de cada sentencia.

Error común: si el script no incluye las comprobaciones, al aplicarlo por segunda vez se producen errores. Se debe usar --idempotent.

Paso 4: Generar un script con un rango de migraciones

```
dotnet ef migrations script InitialCreate AddRowVersion --startup-project ../AceriaData.Console --output migraciones_rango.sql
```

dotnet ef migrations script → genera un script SQL. InitialCreate → migración desde. AddRowVersion → migración hasta. --startup-project ../AceriaData.Console → indica el proyecto de inicio. --output migraciones_rango.sql → guarda el script en el archivo indicado.

Resultado esperado: se crea el archivo migraciones_rango.sql con las migraciones desde InitialCreate hasta AddRowVersion.

Error común: si la migración desde no existe, el comando falla. Se debe verificar el nombre con dotnet ef migrations list.

Paso 5: Generar un script de reversión

```
dotnet ef migrations script AddRowVersion InitialCreate --startup-project ../AceriaData.Console --output reversion.sql
```

dotnet ef migrations script → genera un script SQL. AddRowVersion → migración desde. InitialCreate → migración hasta. --startup-project ../AceriaData.Console → indica el proyecto de inicio. --output reversion.sql → guarda el script en el archivo indicado.

Resultado esperado: se crea el archivo reversion.sql con el script de reversión desde AddRowVersion hasta InitialCreate.

Error común: si se aplica un script de reversión sin copia de seguridad, los datos se pierden. Se debe hacer copia antes.

Paso 6: Verificar los artefactos sin usar tiempos prefijados

La fuente original incluía valores temporales fijos para comparar la generación de scripts. Esos valores no forman parte del contrato canónico porque dependen del equipo, del almacenamiento y del estado de la caché.

Comprueba únicamente que se han generado los tres artefactos esperados y registra, si quieres, el tiempo **observado en esa ejecución** sin convertirlo en una conclusión general.

```
ls -lh migrations.sql migrations-idempotent.sql rollback.sql
```

Corrección técnica aplicada: el criterio de este paso es la existencia y el contenido de los artefactos, no que aparezcan 4200 ms, 4100 ms o 4000 ms.

Paso 7: Revisar el script idempotente

Abrir el archivo migraciones_idempotentes.sql y revisar el contenido. Comprobar las operaciones que se van a ejecutar y los posibles efectos secundarios.

Nota sobre el fragmento SQL de la fuente

El bloque SQL que sigue se conserva para explicar la estructura de un script idempotente, pero es **conceptual y simplificado**: no se presenta como salida exacta de AceriaData. La evidencia del laboratorio es el archivo generado realmente por `dotnet ef migrations script --idempotent`, que debe revisarse y aplicarse dos veces contra la base aislada.

```

IF OBJECT_ID(N'[__EFMigrationsHistory]') IS NULL
BEGIN
    CREATE TABLE [__EFMigrationsHistory] (
        [MigrationId] nvarchar(150) NOT NULL,
        [ProductVersion] nvarchar(32) NOT NULL,
        CONSTRAINT [PK___EFMigrationsHistory] PRIMARY KEY ([MigrationId])
    );
END;
GO

IF NOT EXISTS(SELECT * FROM [__EFMigrationsHistory] WHERE [MigrationId] =
N'20240115120000_InitialCreate')
BEGIN
    CREATE TABLE [OrdenesFabricacion] (
        [Id] int NOT NULL IDENTITY,
        [NumeroOrden] nvarchar(50) NOT NULL,
        [Cliente] nvarchar(200) NOT NULL,
        [FechaCreacion] datetime2 NOT NULL DEFAULT (GETDATE()),
        [RowVersion] rowversion NOT NULL,
        CONSTRAINT [PK_OrdenesFabricacion] PRIMARY KEY ([Id])
    );

    CREATE UNIQUE INDEX [AK_OrdenesFabricacion_NumeroOrden] ON [OrdenesFabricacion] ([NumeroOrden]);
END;
GO

```

Línea 1: IF OBJECT_ID(N'[__EFMigrationsHistory]') IS NULL → comprueba si la tabla de historial existe.

Línea 3: CREATE TABLE [__EFMigrationsHistory] (→ crea la tabla de historial si no existe.

Línea 4: [MigrationId] nvarchar(150) NOT NULL, → columna del identificador de migración.

Línea 5: [ProductVersion] nvarchar(32) NOT NULL, → columna de la versión del producto.

Línea 6: CONSTRAINT [PK___EFMigrationsHistory] PRIMARY KEY ([MigrationId]) → clave primaria.

Línea 11: IF NOT EXISTS(SELECT * FROM [__EFMigrationsHistory] WHERE [MigrationId] = N'20240115120000_InitialCreate') → comprueba si la migración ya está aplicada.

Línea 13: CREATE TABLE [OrdenesFabricacion] (→ crea la tabla si no existe.

Línea 14: [Id] int NOT NULL IDENTITY, → columna Id.

Línea 15: [NumeroOrden] nvarchar(50) NOT NULL, → columna NumeroOrden.

Línea 16: [Cliente] nvarchar(200) NOT NULL, → columna Cliente.

Línea 17: [FechaCreacion] datetime2 NOT NULL DEFAULT (GETDATE()), → columna FechaCreacion con valor por defecto.

Línea 18: [RowVersion] rowversion NOT NULL, → columna RowVersion de tipo rowversion.

Línea 19: CONSTRAINT [PK_OrdenesFabricacion] PRIMARY KEY ([Id]) → clave primaria.

Línea 22: CREATE UNIQUE INDEX [AK_OrdenesFabricacion_NumeroOrden] ON [OrdenesFabricacion] ([NumeroOrden]); → índice único sobre NumeroOrden.

Observaciones: el script idempotente consulta el historial para decidir qué migraciones faltan.

AceriaData conserva la tabla predeterminada `__EFMigrationsHistory` y su historial heredado; 5.7 no renombra ni reinicializa esa cadena.

Paso 8: Aplicar el script idempotente con sqlcmd

```
sqlcmd -S "(localdb)\MSSQLLocalDB" -d AceriaDB -E -i migraciones_idempotentes.sql
```

sqlcmd → invoca la herramienta de línea de comandos de SQL Server. -S "(localdb)\MSSQLLocalDB" → indica la instancia de SQL Server. -d AceriaDB → indica la base de datos de destino. -E → usa autenticación integrada de Windows. -i migraciones_idempotentes.sql → indica el archivo de entrada.

Resultado esperado: el script se aplica a la base de datos AceriaDB. Si el script se aplica de nuevo, no se producen errores porque es idempotente.

Error común: si sqlcmd no está en el PATH, el comando no se reconoce. Se debe instalar la herramienta o usar la ruta completa.

Paso 9: Verificar la aplicación del script

Abrir el Explorador de objetos de SQL Server. Comprobar que las tablas y los índices se han creado correctamente.

```
SELECT * FROM [__EFMigrationsHistory];
```

La primera línea consulta la tabla de historial. Devuelve las migraciones aplicadas.

Resultado esperado: las tablas y los índices aparecen en la base de datos. La tabla de historial contiene las migraciones aplicadas.

Paso 9 bis: Aplicar dos veces el mismo script idempotente y comprobar el historial

La idempotencia debe demostrarse ejecutando realmente el mismo script dos veces contra la misma base de pruebas.

```
sqlcmd -S "(localdb)\MSSQLLocalDB" -d AceriaData_M5_7 -i migrations-idempotent.sql -b
sqlcmd -S "(localdb)\MSSQLLocalDB" -d AceriaData_M5_7 -i migrations-idempotent.sql -b
sqlcmd -S "(localdb)\MSSQLLocalDB" -d AceriaData_M5_7 -Q "SELECT MigrationId FROM __EFMigrationsHistory
ORDER BY MigrationId" -b
```

La segunda aplicación debe terminar correctamente sin duplicar filas en `__EFMigrationsHistory` ni recrear objetos ya migrados. El script idempotente de EF Core consulta el historial de migraciones y aplica únicamente las migraciones pendientes.

Comprueba también que el esquema esperado existe después de ambas ejecuciones. Para los scripts de rango, usa explícitamente una migración `FROM` y una `TO`. Si generas un script con `FROM` posterior a `TO`, trátalo como un **downgrade potencialmente destructivo** y revísalo antes de ejecutarlo.

Paso 10: Crear un pipeline de GitHub Actions simulado

Crear el archivo `.github/workflows/deploy-migrations.yml`:

yaml name: Deploy Migrations

on: push: branches: - main

jobs: deploy: runs-on: ubuntu-latest

```

steps:
  - name: Checkout
    uses: actions/checkout@v4

  - name: Setup .NET
    uses: actions/setup-dotnet@v4
    with:
      dotnet-version: '8.x'

  - name: Install dotnet-ef
    run: dotnet tool install --global dotnet-ef

  - name: Generate idempotent script
    run: dotnet ef migrations script --idempotent --startup-project src/AceriaData.Console --output
migraciones.sql

  - name: Upload script as artifact
    uses: actions/upload-artifact@v4
    with:
      name: migraciones
      path: migraciones.sql

  - name: Apply script to LocalDB
    run: sqlcmd -S "(localdb)\MSSQLLocalDB" -d AceriaDB -E -i migraciones.sql

```

Línea 1: name: Deploy Migrations → nombre del workflow.

Línea 3: on: → define el trigger.

Línea 4: push: → se ejecuta en push.

Línea 5: branches: → ramas que activan el workflow.

Línea 6: - main → la rama principal.

Línea 8: jobs: → define los jobs.

Línea 9: deploy: → nombre del job.

Línea 10: runs-on: ubuntu-latest → sistema operativo del agente.

Línea 12: steps: → define los pasos.

Línea 13: - name: Checkout → primer paso.

Línea 14: uses: actions/checkout@v4 → acción de checkout.

Línea 16: - name: Setup .NET → segundo paso.

Línea 17: uses: actions/setup-dotnet@v4 → acción de instalación de .NET.

Línea 18: with: → parámetros de la acción.

Línea 19: dotnet-version: '8.x' → versión de .NET.

Línea 21: - name: Install dotnet-ef → tercer paso.

Línea 22: run: dotnet tool install --global dotnet-ef → instala la herramienta.

Línea 24: - name: Generate idempotent script → cuarto paso.

Línea 25: run: dotnet ef migrations script --idempotent ... → genera el script.

Línea 27: - name: Upload script as artifact → quinto paso.

Línea 28: uses: actions/upload-artifact@v4 → acción de subida de artefactos.

Línea 29: with: → parámetros.

Línea 30: name: migraciones → nombre del artefacto.

Línea 31: path: migraciones.sql → ruta del archivo.

Línea 33: - name: Apply script to LocalDB → sexto paso.

Línea 34: run: sqlcmd ... -i migraciones.sql → aplica el script.

Error común: si los secretos no están configurados en GitHub, el pipeline falla. Se deben configurar en la configuración del repositorio.

| Paso 11: Crear un pipeline de Azure DevOps simulado

Crear el archivo azure-pipelines.yml:

yaml trigger:

- main

pool: vmImage: 'ubuntu-latest'

steps:

- task: UseDotNet@2 inputs: version: '8.x'
- script: dotnet tool install --global dotnet-ef displayName: 'Instalar dotnet-ef'
- script: dotnet ef migrations script --idempotent --startup-project src/AceriaData.Console --output \$(Build.ArtifactStagingDirectory)/migraciones.sql displayName: 'Generar script idempotente'
- task: PublishBuildArtifacts@1 inputs: PathtoPublish: '\$(Build.ArtifactStagingDirectory)/migraciones.sql' ArtifactName: 'migraciones'
- script: sqlcmd -S "\$(SqlServer)" -d AceriaDB -U "\$(SqlUser)" -P "\$(SqlPassword)" -i \$(Build.ArtifactStagingDirectory)/migraciones.sql displayName: 'Aplicar script' Línea 1: trigger: → define el trigger.

Línea 2: - main → la rama principal.

Línea 4: pool: → define el agente.

Línea 5: vmImage: 'ubuntu-latest' → sistema operativo.

Línea 7: steps: → define los pasos.

Línea 8: - task: UseDotNet@2 → instala .NET.

Línea 9: inputs: → parámetros.

Línea 10: version: '8.x' → versión de .NET.

Línea 12: - script: dotnet tool install --global dotnet-ef → instala la herramienta.

Línea 13: displayName: 'Instalar dotnet-ef' → nombre del paso.

Línea 15: - script: dotnet ef migrations script --idempotent ... → genera el script.

Línea 16: displayName: 'Generar script idempotente' → nombre del paso.

Línea 18: - task: PublishBuildArtifacts@1 → publica el artefacto.

Línea 19: inputs: → parámetros.

Línea 20: PathtoPublish: ... → ruta del archivo.

Línea 21: ArtifactName: 'migraciones' → nombre del artefacto.

Línea 23: - script: sqlcmd ... → aplica el script.

Línea 24: displayName: 'Aplicar script' → nombre del paso.

Error común: si las credenciales no están configuradas como variables secretas, el pipeline falla. Se deben configurar en Azure DevOps.

| Paso 12: Aplicar el script en múltiples bases de datos

Crear el archivo aplicar_multiples_bd.sh:

```
#!/bin/bash

echo "=== APLICAR SCRIPT EN MÚLTIPLES BASES DE DATOS ==="

for db in AceriaDB_Dev AceriaDB_Test AceriaDB_Prod
do
    echo "Aplicando script en $db..."
    sqlcmd -S "(localdb)\MSSQLLocalDB" -d $db -E -i migraciones_idempotentes.sql
    if [ $? -eq 0 ]; then
        echo "OK: $db"
    else
        echo "ERROR: $db"
    fi
done
```

Línea 1: #!/bin/bash → shebang del script.

Línea 3: echo "=== APLICAR SCRIPT EN MÚLTIPLES BASES DE DATOS ===" → muestra la cabecera.

Línea 5: for db in AceriaDB_Dev AceriaDB_Test AceriaDB_Prod → itera sobre las bases de datos.

Línea 6: do → inicio del bloque.

Línea 7: echo "Aplicando script en \$db..." → muestra el mensaje.

Línea 8: sqlcmd ... -d \$db -E -i migraciones_idempotentes.sql → aplica el script.

Línea 9: if [\$? -eq 0]; then → comprueba si el comando ha tenido éxito.

Línea 10: echo "OK: \$db" → muestra el mensaje.

Línea 11: else → caso contrario.

Línea 12: echo "ERROR: \$db" → muestra el error.

Línea 13: fi → cierre del condicional.

Línea 14: done → cierre del bucle.

Error común: si alguna base de datos no existe, el comando falla. Se debe verificar la existencia antes de aplicar.

| Paso 13: Ejecutar el script de múltiples bases de datos

```
chmod +x aplicar_multiples_bd.sh
./aplicar_multiples_bd.sh
```

Resultado esperado: el script se aplica a las tres bases de datos. Si alguna no existe, se muestra un error.

| Paso 14: Analizar la salida

La salida del comando muestra información como la siguiente:

=== APLICAR SCRIPT EN MÚLTIPLES BASES DE DATOS ===

Aplicando script en AceriaDB_Dev...

OK: AceriaDB_Dev

Aplicando script en AceriaDB_Test...

OK: AceriaDB_Test

Aplicando script en AceriaDB_Prod...

ERROR: AceriaDB_Prod

La primera sección muestra la cabecera. La segunda sección aplica el script en las bases de datos. La tercera sección muestra los resultados.

Observaciones: el script se aplica correctamente en las bases de datos que existen. La base de datos que no existe muestra un error.

Paso 15: Diagnosticar un error común

Modificar el script migraciones_idempotentes.sql para eliminar las comprobaciones IF NOT EXISTS:

```
CREATE TABLE [OrdenesFabricacion] (
    [Id] int NOT NULL IDENTITY,
    [NumeroOrden] nvarchar(50) NOT NULL,
    [Cliente] nvarchar(200) NOT NULL,
    [FechaCreacion] datetime2 NOT NULL DEFAULT (GETDATE()),
    [RowVersion] rowversion NOT NULL,
    CONSTRAINT [PK_OrdenesFabricacion] PRIMARY KEY ([Id])
);
GO
```

Resultado esperado: al aplicar el script por segunda vez, se produce un error porque la tabla ya existe.

Solución: restaurar las comprobaciones IF NOT EXISTS.

```
IF NOT EXISTS(SELECT * FROM [__EFMigrationsHistory] WHERE [MigrationId] =
N'20240115120000_InitialCreate')
BEGIN
    CREATE TABLE [OrdenesFabricacion] (...);
END;
GO
```

Resultado esperado con la solución: el script se puede aplicar varias veces sin error.

Errores comunes del ejercicio

Error Causa Solución Script no idempotente Se aplicó varias veces Usar --idempotent Migración desde no existe El nombre es incorrecto Verificar con dotnet ef migrations list Script elimina datos El script elimina una columna con datos Revisar el script y hacer copia de seguridad Script aplicado en la base equivocada No se verificó la base de datos de destino Verificar la base de datos antes de aplicar sqlcmd no reconocido No está en el PATH Instalar la herramienta o usar la ruta completa Pipeline aplica sin revisar No se revisó el script Revisar antes de aplicar Autenticación de Windows falla El usuario no tiene permisos Otorgar los permisos necesarios

Reto resuelto: Integrar scripts en un pipeline de GitHub Actions con secretos

Reto: Crear un pipeline de GitHub Actions que genere el script idempotente, lo suba como artefacto y lo aplique en una base de datos SQL Server remota usando secretos. Simular el flujo con un pipeline local.

Solución paso a paso:

Paso 1: Crear el archivo .github/workflows/deploy-migrations.yml:

yaml name: Deploy Migrations

on: push: branches: - main

jobs: deploy: runs-on: ubuntu-latest

```
steps:
  - name: Checkout
    uses: actions/checkout@v4

  - name: Setup .NET
    uses: actions/setup-dotnet@v4
    with:
      dotnet-version: '8.x'

  - name: Install dotnet-ef
    run: dotnet tool install --global dotnet-ef

  - name: Generate idempotent script
    run: dotnet ef migrations script --idempotent --startup-project src/AceriaData.Console --output
migraciones.sql

  - name: Upload script as artifact
    uses: actions/upload-artifact@v4
    with:
      name: migraciones
      path: migraciones.sql

  - name: Apply script to SQL Server
    run: sqlcmd -S "${{ secrets.SQL_SERVER }}" -d "${{ secrets.SQL_DATABASE }}" -U "$
${{ secrets.SQL_USER }}" -P "${{ secrets.SQL_PASSWORD }}" -i migraciones.sql
```

Línea 1: name: Deploy Migrations → nombre del workflow.

Línea 3: on: → define el trigger.

Línea 4: push: → se ejecuta en push.

Línea 5: branches: → ramas que activan el workflow.

Línea 6: - main → la rama principal.

Línea 8: jobs: → define los jobs.

Línea 9: deploy: → nombre del job.

Línea 10: runs-on: ubuntu-latest → sistema operativo del agente.

Línea 12: steps: → define los pasos.

Línea 13: - name: Checkout → primer paso.

Línea 14: uses: actions/checkout@v4 → acción de checkout.

Línea 16: - name: Setup .NET → segundo paso.

Línea 17: uses: actions/setup-dotnet@v4 → acción de instalación de .NET.

Línea 18: with: → parámetros de la acción.

Línea 19: dotnet-version: '8.x' → versión de .NET.

Línea 21: - name: Install dotnet-ef → tercer paso.

Línea 22: run: dotnet tool install --global dotnet-ef → instala la herramienta.

Línea 24: - name: Generate idempotent script → cuarto paso.

Línea 25: run: dotnet ef migrations script --idempotent ... → genera el script.

Línea 27: - name: Upload script as artifact → quinto paso.

Línea 28: uses: actions/upload-artifact@v4 → acción de subida.

Línea 29: with: → parámetros.

Línea 30: name: migraciones → nombre del artefacto.

Línea 31: path: migraciones.sql → ruta del archivo.

Línea 33: - name: Apply script to SQL Server → sexto paso.

Línea 34: run: sqlcmd ... -i migraciones.sql → aplica el script con secretos.

Paso 2: Configurar los secretos en GitHub:

1. Ir al repositorio en GitHub.

1. Ir a Settings → Secrets and variables → Actions.

2. Crear los secretos: SQL_SERVER, SQL_DATABASE, SQL_USER, SQL_PASSWORD.

Paso 3: Hacer push a la rama principal y verificar que el pipeline se ejecuta.

Resultado esperado: el pipeline genera el script idempotente, lo sube como artefacto y lo aplica en la base de datos remota con los secretos.

Analogía final

Las migraciones idempotentes en una acería son como los manuales de reforma que se pueden aplicar varias veces sin riesgo. Si la reforma ya está hecha, el manual no la repite. Los scripts con rango de migraciones son como los manuales que solo cubren un tramo de la reforma. Los scripts de reversión son como los manuales que deshacen la reforma.

Aplicar los scripts con sqlcmd es como aplicar el manual en la planta con las herramientas adecuadas. La autenticación integrada de Windows es como usar la llave maestra de la planta. La autenticación de SQL Server es como usar una llave específica para una puerta. Los permisos restringidos son como las llaves limitadas que solo abren ciertas puertas.

Integrar los scripts en CI/CD es como automatizar la aplicación de los manuales en cada planta. El pipeline de GitHub Actions es como una cadena de montaje que genera el manual, lo revisa, lo archiva y lo aplica. Los secretos son como las llaves que se guardan en una caja fuerte. La revisión previa es como leer el manual antes de aplicarlo.

El impacto de los scripts en el rendimiento es de milisegundos. La generación de un script tarda unos 4 segundos. La aplicación depende del número de migraciones y del tamaño de las tablas.

Así funcionan las migraciones idempotentes en EF Core: se generan, se revisan y se aplican de forma controlada en cada entorno.

Resultado esperado

Al final del ejercicio, deberías haber:

Generado un script con todas las migraciones.

Generado un script idempotente.

Generado un script con un rango de migraciones.

Generado un script de reversión.

Medido el tiempo de generación de cada script.

Revisado el script idempotente.

Aplicado el script idempotente con sqlcmd con autenticación integrada.

Verificado la aplicación del script.

Creado un pipeline de GitHub Actions.

Creado un pipeline de Azure DevOps.

Aplicado el script en múltiples bases de datos.

Diagnosticado el error de eliminar las comprobaciones IF NOT EXISTS.

| Conexión con el siguiente punto

En este punto se han estudiado las migraciones idempotentes y los scripts SQL, incluyendo su generación, revisión, aplicación y uso en pipelines de CI/CD. Se ha comprobado que los scripts idempotentes se pueden aplicar varias veces sin error y que son esenciales para desplegar migraciones de forma controlada. También se han cubierto los pipelines de Azure DevOps y GitHub Actions, la autenticación integrada de Windows y de SQL Server, y el uso de scripts en entornos con múltiples bases de datos.

En el siguiente punto se estudiarán las migraciones en equipos: conflictos, merges y buenas prácticas, con sus implicaciones en el trabajo colaborativo.
